

Filebeat · Logstash · Grok

ZH felkészítő ■ összefoglaló

Ez az összefoglaló a három legfontosabb témakört fedi le: a Filebeat által figyelt logfájlokat, a teljes Logstash pipeline konfigurációt grok filterrel, és a **date filter** részletes magyarázatát.

1. Milyen logfájlokat kell figyelni Filebeattal?

Az alapelv

A Filebeat **bármilyen szöveges logfájlt** tud olvasni. A ZH-n megmondják, melyik konkrét fájlt kell figyelni. A filebeat.yml-be egyszer ■ en megadod az elérési utat:

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /var/log/apache2/access.log
```

Leggyakoribb logfájlok ZH-n

Web szerver logok

```
paths:
- /var/log/nginx/access.log
- /var/log/nginx/error.log
- /var/log/apache2/access.log
- /var/log/apache2/error.log
```

Ezek a legvalószínűbbek, mert rájuk léteznek kész grok minták (pl. COMBINEDAPACHELOG).

Syslog / rendszerlogok

```
paths:
- /var/log/syslog
- /var/log/auth.log      # SSH bejelentkezések, sudo
- /var/log/kern.log
```

Alkalmazás logok (egyedi formátum)

```
paths:
- /var/log/myapp/application.log
- /opt/myapp/logs/*.log    # glob minta
```

Amire figyelj a konfigurációban

1. Az **enabled: true** ne maradjon ki — egyes verziókban alapértelmezés szerint false.

2. Glob minták használhatók:

```
paths:
```

- /var/log/nginx/*.log # minden nginx log
- /var/log/**/*.log # rekurzívan minden log

3. Több logforrás esetén tags-szel különböztess meg őket:

```
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /var/log/nginx/access.log
  tags: ["nginx", "access"]

- type: log
  enabled: true
  paths:
    - /var/log/auth.log
  tags: ["auth", "syslog"]
```

Logstash-ben aztán így szűrsz rájuk:

```
filter {
  if "nginx" in [tags] {
    grok {
      match => { "message" => "%{COMBINEDAPACHELOG}" }
    }
  }
  if "auth" in [tags] {
    grok {
      match => { "message" => "%{SYSLOGLINE}" }
    }
  }
}
```

ZH-s tanács

A feladat leírása valószínűleg megmondja a pontos elérési utat. Ellenőrizd, hogy a fájl létezik-e és Filebeatnek van-e olvasási joga:

```
ls -la /var/log/nginx/access.log
sudo usermod -aG adm filebeat # ha nincs jog
```

2. Logstash pipeline konfiguráció és Grok filter

A pipeline három blokkja

Minden Logstash konfiguráció (/etc/logstash/conf.d/pipeline.conf) három kötelező blokkból áll:

```
input {
  beats {
    port => 5044 # Filebeat-től fogad ezen a porton
  }
}
```

```

filter {
  grok { ... }          # szöveg → strukturált mezők
  date { ... }          # időbélyeg kezelés
  mutate { ... }        # mezők módosítása, törlése
}

output {
  opensearch {
    hosts => ["https://localhost:9200"]
    index => "weblog-%{+YYYY.MM.dd}"
    user => "admin"
    password => "admin"
    ssl_certificate_verification => false
  }
}

```

A Grok filter

A grok regex-alapú parser, de olvashatóbb szintaxissal. A nyers log szövegből nevesített mezőket csinál. Szintaxisa:

```
%{MINTA_NEVE:mez_neve}
```

Beépített minták — legfontosabbak

Minta	Mit fog meg	Példa
%{IP}	IPv4 cím	192.168.1.1
%{NUMBER}	szám (int/float)	200, 3.14
%{WORD}	szóköz nélküli szó	GET, ERROR
%{URIPATHPARAM}	URL path + query	/api/v1?id=5
%{HTTPDATE}	HTTP dátumformátum	10/Oct/2024:13:55:36 +0000
%{TIMESTAMP_ISO8601}	ISO dátum	2024-10-10T13:55:36
%{LOGLEVEL}	log szint	ERROR, WARN, INFO
%{GREEDYDATA}	mindent megfog	(maradék sor)
%{COMBINEDAPACHELOG}	teljes Apache/Nginx sor	(egyben mindent)

Nginx access log parsing — teljes példa

```

filter {
  grok {
    match => { "message" => "%{COMBINEDAPACHELOG}" }
  }
}

# Eredmény mezők:
# clientip, ident, auth, timestamp, verb,
# request, httpversion, response, bytes

```

Saját minta írása

Ha a beépített minták nem elégek, saját regex-et írhatasz:

```
filter {
  grok {
    match => { "message" => "(?<error_code>ERR-\d+) %{GREEDYDATA:description}" }
  }
}
```

Több alternatív minta

Ha a logok nem egységes formátumúak, listában add meg a mintákat — Logstash sorban próbálja mindegyiket:

```
grok {
  match => {
    "message" => [
      "%{COMBINEDAPACHELOG}",      # először ezt próbálja
      "%{COMMONAPACHELOG}",        # ha nem jön össze, ezt
      "%{GREEDYDATA:raw_message}"  # végső fallback
    ]
  }
}
```

Debug tippek

Filebeat logok:

```
sudo journalctl -u filebeat -f
```

Logstash konfiguráció tesztelése:

```
/usr/share/logstash/bin/logstash -f /etc/logstash/conf.d/pipeline.conf --config.test_and_exit
```

Ha a grok nem parsolja a sort, _grokparsefailure tag kerül az eseményre — a message mező érintetlen marad. Teszteld a Grok Debuggerben!

3. A date filter részletes magyarázata

Miért kell a date filter?

Amikor egy log beérkezik Logstash-be, az @timestamp mező **automatikusan a beérkezés pillanatát** kapja. Ez helytelen, mert ha régebbi logfájlt töltesz be, minden esemény 'most' keletkezett látszólag. A date filter a log szövegéből kiveszi a valódi időbélyeget és azt teszi @timestamp-be.

A konfiguráció

```
date {
  match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z", "ISO8601" ]
  target => "@timestamp"
  remove_field => "timestamp"
}
```

Sor by sor magyarázat

match => ["timestamp", "dd/MMM/yyyy:HH:mm:ss Z", "ISO8601"]

Ez egy lista **három elemmel** — de nem három mező, hanem **egy mező + több formátum**:

- "timestamp" — a mező neve, ahova a grok filter korábban kieszedta az időbélyeget a log szövegéből.
- "dd/MMM/yyyy:HH:mm:ss Z" — az Apache/Nginx access log formátuma, pl.
10/Oct/2024:13:55:36 +0000
- "ISO8601" — ha az első nem illeszkedik, ezt próbálja: 2024-10-10T13:55:36

Annyi formátumot sorolhatsz fel, amennyit csak akarsz. Logstash sorban próbálja mindegyiket.

Dátumformátum szimbólumok

Szimbólum	Jelentés	Példa
dd	nap, 2 jegy	05, 31
MM	hónap, szám	10
MMM	hónap, szöveg	Oct
yyyy	év, 4 jegy	2024
HH	óra (0–23)	13
mm	perc	55
ss	másodperc	36
Z	timezone offset	+0000

target => "@timestamp"

Ide kerül a feldolgozott időbélyeg. Az @timestamp az OpenSearch speciális mezője — ez alapján rendezi időrendbe az eseményeket, ez jelenik meg a dashboardon az X tengelyen, és ez alapján szűrheted időintervallumra.

remove_field => "timestamp"

Ha ezt kihagynád, az eseményen **két** időbélyeg mező lenne:

- @timestamp — amit most állítottunk be (helyes)
- timestamp — az eredeti szöveges érték a grokból (redundáns)

Ezért töröljük — felesleges adatot tárolna, és összezavarhat kereséskor.

Az egész folyamat egy példán

Tegyük fel, ez a nyers log sor érkezik be:

```
192.168.1.1 - - [10/Oct/2024:13:55:36 +0000] "GET /index.html HTTP/1.1" 200 512
```

1. lépés — grok kiveszi a mezőket, köztük:

```
timestamp = "10/Oct/2024:13:55:36 +0000"
```

2. lépés — date feldolgozza a timestamp mezőt:

@timestamp = 2024-10-10T13:55:36.000Z ← ez kerül OpenSearch-be

3. lépés — remove_field után:

timestamp mező → törölve

Ami könnyen elromolhat

Ha a dátumformátum nem pontosan egyezik a log szövegével, a date filter csendben **nem csinál semmit** — az @timestamp marad a beérkezési idő. Ezt úgy veszed észre, hogy az OpenSearch-ben minden esemény 'most' keletkezett. Ilyenkor a Grok Debuggerben ellenőrizd, hogy a timestamp mező valóban létrejön-e, és a formátum pontosan stimmel-e.